

COMP 0089 Reinforcement Learning.

1. Return $G_t = R_{t+1} + R_{t+2} + \dots$

2. Value: Expected cumulative reward from a state s $V(s) = \mathbb{E}[G_t | S_t = s]$
picking suitable actions to maximise value.

3. Recursively: $G_t = R_{t+1} + G_{t+1}$, $V(s) = \mathbb{E}[R_{t+1} + V(S_{t+1}) | S_t = s]$

4. Value on actions: $Q(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$

5. Markov decision process (MDPs)

A decision process is Markov if $P(r, s' | S_t, A_t) = P(r, s' | \mathcal{H}_t, A_t)$.
↑
full history.

S_t is agent state, not environment state. Agent state is Markovian. Not observation or env.

6. With discount factor: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = R_{t+1} + \gamma G_{t+1}$.

Bellman equation: $V_\pi(s) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = s, A_t \sim \pi(s)] = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = s, A_t \sim \pi(s)]$

optimal values: $V_*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma V_*(S_{t+1}) | S_t = s, A_t = a]$

7. Model: predicts what environment will do next. $P(s', a' | s, a) \approx P(S_{t+1} = s' | S_t = s, A_t = a)$

$Q(s, a) \approx \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$

8. Multi-armed Bandit: action value for action a : $Q(a) = \mathbb{E}[R_t | A_t = a]$

optimal value

$V_* = \max_{a \in A} Q(a) = \max_a \mathbb{E}[R_t | A_t = a]$

regret for an action a : $\Delta_a = V_* - Q(a)$

We want to minimize total regret: $L_t = \sum_{n=1}^t V_* - Q(A_n) = \sum_{n=1}^t \Delta_{A_n}$

maximise cumulative reward $\equiv$ minimize total regret.

9. Estimate for action value: $Q_t(a) = \frac{\sum_{n=1}^t I(A_n = a) R_n}{\sum_{n=1}^t I(A_n = a)}$, $N_t(a) = \sum_{n=1}^t I(A_n = a)$

$$Q_t(A_t) = \frac{N_{t-1}(A_t)}{N_t(A_t)} Q_{t-1}(A_t) + \frac{1}{N_t(A_t)} R_t = \frac{N_{t-1}(A_t) - 1}{N_t(A_t)} Q_{t-1}(A_t) + \frac{1}{N_t(A_t)} R_t$$

$$= Q_{t-1}(A_t) + \frac{1}{N_t(A_t)} (R_t - Q_{t-1}(A_t))$$

$$= Q_{t-1}(A_t) + a_t (R_t - Q_{t-1}(A_t))$$

$\forall a \neq A_t, Q_t(a) = Q_{t-1}(a)$

10. Greedy: select action with highest value: $A_t = \arg\max_a Q_t(a)$

11. ϵ -greedy: $1 - \epsilon$ probability select greedy action: $a = \arg\max_{a \in A} Q_t(a)$

constant ϵ has ϵ probability select random

linear expected total regret. $\pi_t(a) = \begin{cases} (1-\epsilon) + \epsilon/|A| & \text{if } Q_t(a) = \max_b Q_t(b) \\ \epsilon/|A| & \text{otherwise} \end{cases}$

$$R_T = \sum_{t=1}^T (V^* - Q(A_t)) = \sum_{i \neq a^*} \Delta_i N_i(T), \quad \mathbb{E}[V^* - Q(A_t)] \geq p_{\text{sub}} \Delta_{\min} \geq \epsilon \frac{K-1}{K} \Delta_{\min}$$

assume $(1-\epsilon)$ prob. we pick the optimal

$$P(A_t \text{ is suboptimal} | \text{exploration}) = \frac{K-1}{K}$$

$$\mathbb{E}[R_T] \geq \epsilon \frac{K-1}{K} \Delta_{\min} T$$

12. Policy gradients: eg. Softmax of action preferences $\pi(a) = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}}$

gradient ascent $Q_{t+1} = Q_t + \alpha \nabla_{\theta} \mathbb{E}[R_t | \pi_{\theta}]$, θ_t is current policy parameters.

$$\nabla_{\theta} \mathbb{E}[R_t | \pi_{\theta}] = \nabla_{\theta} \sum_a \pi_{\theta}(a) Q(a) = \sum_a Q(a) \nabla_{\theta} \pi_{\theta}(a) = \sum_a Q(a) \pi_{\theta}(a) \nabla \log \pi_{\theta}(a) = \mathbb{E}_{\pi} [R_t \nabla \log \pi_{\theta}(A_t)]$$

$Q_{t+1}^{(b)} = Q_t^{(b)} + \alpha R_t \nabla_{\theta} \log \pi_{\theta}(A_t)$ stochastic gradient ascent

For softmax: $H_{t+1}(a) = H_t(a) + \alpha R_t \frac{\partial \log \pi_t(a)}{\partial H_t(a)} = H_t(a) + \alpha R_t (I(a = A_t) - \pi_t(a))$

$$H_{t+1}(A_t) = H_t(A_t) + \alpha R_t (1 - \pi_t(A_t))$$

If $R_t > 0$, $H_{t+1}(A_t) \uparrow$, $H_{t+1}(a) \downarrow$

$$H_{t+1}(a) = H_t(a) - \alpha R_t \pi_t(a) \quad \text{if } a \neq A_t$$

For any b , $\sum_a b \nabla_{\theta} \pi_{\theta}(a) = b \nabla_{\theta} \sum_a \pi_{\theta}(a) = b \nabla_{\theta} 1 = 0$

so we can have $\theta_{t+1} = \theta + \alpha(R_t - b) \nabla_{\theta} \log \pi_{\theta}(A_t) \Rightarrow$ Expected value doesn't change but variance does.

13. Lai and Robbins: Asymptotic total regret is at least logarithm in number of steps

$$\lim_{t \rightarrow \infty} L_t \geq \log t \sum_{a \neq a^*} \frac{\Delta_a}{KL(R_a || R_{a^*})} \quad KL(R_a || R_{a^*}) \propto \Delta_a^2 \quad \Delta_a = V_a - q(a)$$

14. $L_t = \sum_{n=1}^t \Delta_{A_n} = \sum_{a \in A} N_t(a) \Delta_a$ A good algorithm ensures small counts for large action regrets.

15. Optimism in the face of uncertainty: choose $\max_a (\mu_a + c \sigma_a)$, the highest upper bound.

- discover new arm if it is optimism
- reduce uncertainty if not optimism.

16. Upper confidence bound (UCB): $A_t = \arg \max_{a \in A} Q_t(a) + U_t(a)$ $q(a) \leq Q_t(a) + U_t(a)$ with high prob.

- small $N_t(a) \Rightarrow$ (large $U_t(a)$) Q_t is sample mean, q is true mean
- a is only selected either $Q_t(a)$ is large (good action) or $U_t(a)$ is large (high uncertainty)

17. Hoeffding's inequality: $X_1, \dots, X_n$ be i.i.d random variables in $[0, 1]$, $\mu = \mathbb{E}[X]$

let $\bar{X}_t = \frac{1}{n} \sum_{i=1}^n X_i$ be the sample mean. Then $P(\bar{X}_n + u \leq \mu) \leq e^{-2nu^2}$

18. Select $U_t(a)$ such that $P(Q_t(a) + U_t(a) \leq q(a)) \leq e^{-2N_t(a)U_t(a)^2} = p$

19. UCB: $A_t = \arg \max_{a \in A} Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}}$ when we select action, $N_t(a) \uparrow$, $U_t(a) \downarrow$
when we don't select, $U_t(a)$ grows in $\sqrt{\log t}$

20. UCB with $c = \sqrt{2}$ achieves logarithmic expected total regret.

$$L_t \leq 8 \sum_{a \neq a^*} \frac{\log t}{\Delta_a} + O(\sum_a \Delta_a) \quad \forall t$$

21. Bayesian Bandits with upper confidence bounds: we estimate upper confidences from the posterior.

$U_t(a) = c \sigma_t(a)$ where $\sigma(a)$ is standard deviation of $p_t(q(a))$

$q(a)$ means the expected value of reward. For a Bernoulli type reward, $q(a) = \Pr(\text{reward} = 1) = p$

$p(x|p)$ is the probability of reward. $P(p|X) \propto \prod p(x|p) p(p)$

22. Probability matching: select action a according to the probability (belief) that a is optimal
 $\pi_t(a) = P(q(a) = \max_{a'} q(a') | H_{t+1})$. $P(q(a))$ is posterior probability.

23. Thompson sampling: sample $Q_t(a) \sim p_t(q(a)) \forall a$, select action $A_t = \arg \max_{a \in A} Q_t(a)$
 $\pi_t(a) = \mathbb{E}[I(Q_t(a) = \max_{a'} Q_t(a'))] = P(q(a) = \max_{a'} q(a'))$.

24. MDP: a tuple (S, A, p, r, γ) where S is the set of all possible states. A is set of all possible actions. $p(r, s' | s, a)$ is joint probability of a reward r and next state s' , given a state s and action a . $\gamma \in [0, 1]$ discount factor.

$$p(s' | s, a) = \sum_r p(r, s' | s, a) \quad \mathbb{E}[R | s, a] = \sum_r r \sum_{s'} p(r, s' | s, a)$$

26. Alternative definition: (S, A, p, r, γ) $p(s' | s, a)$ transition probability.

$r: S \times A \rightarrow \mathbb{R}$ the expected reward, achieved on a transition starting in (s, a)

$$r = \mathbb{E}[R | s, a] = \sum_r r \sum_{s'} p(s', r | s, a)$$

27. Goal of an RL agent: find a behaviour policy that maximises the expected return G_t .

28. Policy: mapping $\pi: S \times A \rightarrow [0, 1]$ $\pi(a | s)$.

29. Value function $V(s) = \mathbb{E}[G_t | S_t = s]$ state-action values $Q(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$.

$$V(s) = \sum_a \pi(a | s) Q(s, a) = \mathbb{E}[Q(S_t, A_t) | S_t = s]$$

30. An optimal policy can be found by maximizing over $q^*(S, a)$ - there is always a deterministic optimal policy for any MDP.
 $\pi^*(S, a) = \begin{cases} 1 & \text{if } a = \arg\max_{a \in \mathcal{A}} q^*(S, a) \\ 0 & \text{otherwise} \end{cases}$ - we know $q^*(S, a) \Rightarrow$ we know π^*

31. $V_\pi(S) = \mathbb{E}[G_t | S_t = S] = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = S] = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = S]$
 $= \sum_a \pi(a|S) \sum_{r, S'} p(r, S' | S, a) (r + \gamma V_\pi(S'))$

32. $Q_\pi(S, a) = \mathbb{E}[G_t | S_t = S, A_t = a] = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = S, A_t = a] = \mathbb{E}[R_{t+1} + \gamma \mathbb{E}[Q_\pi(S_{t+1}, A_{t+1}) | S, a]]$
 $= \sum_r \sum_{S'} p(r, S' | S, a) (r + \gamma \sum_{a'} \pi(a' | S') Q_\pi(S', a'))$

33. Bellman Expectation Equations: Given an MDP, $\mathcal{M} = \langle S, \mathcal{A}, p, r, \gamma \rangle$, for $\forall \pi$, the value functions obey:

$$V_\pi(S) = \sum_a \pi(a|S) [r(S, a) + \gamma \sum_{S'} p(S'|S, a) V_\pi(S')] \\ Q_\pi(S, a) = r(S, a) + \gamma \sum_{S'} p(S'|S, a) \sum_{a'} \pi(a' | S') Q_\pi(S', a')$$

34. Bellman Optimality Equations: $V^*(S) = \max_a [r(S, a) + \gamma \sum_{S'} p(S'|S, a) V^*(S')]$
 $Q^*(S, a) = r(S, a) + \gamma \sum_{S'} p(S'|S, a) \max_{a'} Q_\pi(S', a')$

35. Policy evaluation: estimating V_π or Q_π given a π .

Policy optimisation (control): optimise policy and optimal value function.

36. $V_\pi(S) = \sum_a \pi(a|S) r(S, a) + \gamma \sum_a \sum_{S'} p(S'|S, a) \pi(a|S) V_\pi(S')$
 $= \mathbb{E}[r(S, a) | S] + \gamma \sum_{S'} V_\pi(S') \sum_a p(S'|S, a) \pi(a|S) \Rightarrow V = r^\pi + \gamma P^\pi V$
 $= \mathbb{E}[R_{t+1} | S] + \gamma \sum_{S'} V_\pi(S') p(S'|S)$ $V_i = V(S_i) \quad r_i^\pi = \mathbb{E}[R_{t+1} | S_i]$
 $P_{ij}^\pi = p(S' = S_j | S = S_i)$

$$V = (I - \gamma P^\pi)^{-1} r^\pi$$

37. Policy evaluation: estimate $V_\pi(S) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = S]$
 First, initialize V_0 , Iterate $\forall S: V_{k+1}(S) \leftarrow \mathbb{E}[R_{t+1} + \gamma V_k(S_{t+1}) | S, \pi]$
 stop whenever $V_{k+1}(S) = V_k(S)$.

38. Policy Improvement: Iterate, using $\forall S: \pi_{\text{new}}(S) = \arg\max_a Q_\pi(S, a)$
 $= \arg\max_a \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = S, A_t = a]$

39. $V^*(S) = \max_{a \in \mathcal{A}} Q^*(S, a)$, for $\forall a: Q^*(S, a) \leq V^*(S)$ $V^*(S)$ is the return in S if we always act optimally
 $Q^*(S, a^*) = V^*(S) \quad a^* = \arg\max_a Q^*(S, a)$ $Q^*(S, a)$ is the return if we start from a specific a and act optimally thereafter.

40. $Q_\pi(S, a) = r(S, a) + \gamma \mathbb{E}_{S', a'} [Q_\pi(S', a')] \leq r(S, a) + \gamma \mathbb{E}_{S'} [\max_{a'} Q_\pi(S', a')]$ $\pi_{\text{new}} = \delta(a = \arg\max_{a'} Q_\pi(S, a'))$
 $= r(S, a) + \gamma \mathbb{E}_{S'} [Q_\pi(S', \pi_{\text{new}}(S'))]$
 $= r(S, a) + \gamma \mathbb{E}_{S', a' \sim \pi_{\text{new}}} [Q_\pi(S', a')]$
 $= Q_{\pi_{\text{new}}}(S, a)$
 $V_\pi(S) = \sum_a \pi(a|S) Q_\pi(S, a) \leq \max_a Q_\pi(S, a) \leq \sum_a \pi_{\text{new}}(a|S) Q_{\pi_{\text{new}}}(S, a) = \sum_a \delta(a = \arg\max_{a'} Q_\pi(S, a')) Q_{\pi_{\text{new}}}(S, a)$
 $= Q_{\pi_{\text{new}}}(S, a^*) = V_{\text{new}}(S)$

41. Value iteration: equivalent to policy iteration, with $k=1$ step of policy evaluation
 Initialize V_0 , update $V_{k+1}(S) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma V_k(S_{t+1}) | S_t = S]$
 stop whenever $V_{k+1}(S) = V_k(S)$ for $\forall S$.

42. Problem	Which Bellman?	Algorithm.	Complexity on state-value function
Prediction	Expectation	Policy evaluation	$V_\pi(S) \Rightarrow O(\mathcal{A} \mathcal{S} ^2)$ per iteration
Control	Expectation + greedy policy improvement	Policy Iteration,	complexity on $Q(S, a) \Rightarrow O(\mathcal{A} ^2 \mathcal{S} ^2)$
control	Optimality	Value Iteration	

43. $\mathcal{X}$ be vector space, equipped with a norm $\|\cdot\|$. A mapping $T: \mathcal{X} \rightarrow \mathcal{X}$ is a α -contraction mapping if $\forall x_1, x_2 \in \mathcal{X}, \exists \alpha \in [0, 1)$ s.t. $\|Tx_1 - Tx_2\| \leq \alpha \|x_1 - x_2\|$

If $\alpha \in [0, 1)$, T is non-expanding. It is also Lipschitz and also continuous.

44. Fixed point: $x \in \mathcal{X}$ is a fixed point of an operator T , if $Tx = x$

45. Banach fixed point theorem: $\mathcal{X}$ be complete normed vector space, equipped with a norm $\|\cdot\|$ and $T: \mathcal{X} \rightarrow \mathcal{X}$ a γ -contraction mapping, then:

① T has a unique fixed point $x^* \in \mathcal{X}$: $\exists! x^* \in \mathcal{X}$ s.t. $Tx^* = x^*$

② $\forall x_0 \in \mathcal{X}$, the sequence $x_{n+1} = Tx_n$ converges to x^* in a geometric fashion:

$$\|x_n - x^*\| \leq \gamma^n \|x_0 - x^*\|, \text{ thus } \lim_{n \rightarrow \infty} \|x_n - x^*\| \leq \lim_{n \rightarrow \infty} (\gamma^n \|x_0 - x^*\|) = 0.$$

46. Bellman Optimality Operator T_V^* : let $\mathcal{V} \equiv \mathcal{V}_S$ be the space of bounded real-valued function over S . We define, point-wise, the Bellman Optimality operator $T_V^*: \mathcal{V} \rightarrow \mathcal{V}$ as:

$$(T_V^* f)(\cdot) = \max_a [r(\cdot, a) + \gamma \sum_{s'} p(s'|a, \cdot) f(s')] \quad \forall f \in \mathcal{V}.$$

- It has one unique fixed point v^* .

- T^* is a γ -contraction wrt $\|\cdot\|_\infty$ $\|T^*v - T^*u\|_\infty \leq \gamma \|v - u\|_\infty \quad \forall u, v \in \mathcal{V}$. γ -discount factor.

- T^* is monotonic: $\forall u, v \in \mathcal{V}$ s.t. $u \leq v$, component-wise, then $T^*u \leq T^*v$.

47. Bellman Expectation Operator T_V^π : let $\mathcal{V} \equiv \mathcal{V}_S$ be the space of bounded real-valued functions over S . For any policy $\pi: S \times A \rightarrow [0, 1]$, we define point-wise Bellman Expectation operator $T_V^\pi: \mathcal{V} \rightarrow \mathcal{V}$ as:

$$(T_V^\pi f)(\cdot) = \sum_a \pi(\cdot, a) [r(\cdot, a) + \gamma \sum_{s'} p(s'|a, \cdot) f(s')]. \quad \forall f \in \mathcal{V}$$

Same properties as T^* above.

48. For q^* and q^π , we have $\mathcal{Q} \equiv \mathcal{Q}_{S,A}$ space.

$$(T_{\mathcal{Q}}^* f)(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|s, a) \sum_{a'} \pi(a'|s') f(s', a'), \quad \forall f \in \mathcal{Q}$$

$$(T_{\mathcal{Q}}^\pi f)(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|s, a) \max_{a'} f(s', a') \quad \forall f \in \mathcal{Q}$$

Same properties as T^* and T^π

49. Approximate Value Iteration: start with v_0 , update values: $v_{k+1} = T^* v_k$

using a function approximator $V_\theta(s)$, with a parameter vector $\theta \in \mathbb{R}^n$.

$$V_k = V_{\theta_k}, \text{ calculate } V_{\theta_{k+1}} \text{ from } V_{\theta_k}: T^* V_k(s) = \max_a \mathbb{E}[R_{t+1} + \gamma V_k(S_{t+1}) | S_t = s]$$

$$\text{fit } \theta_{k+1}, \text{ s.t. } V_{\theta_{k+1}} \approx T^* V_k(s) \quad \text{eg: } \theta_{k+1} = \arg \min_{\theta} \sum_s (V_{\theta_{k+1}}(s) - T^* V_k(s))^2.$$

50. Value of greedy policy: consider an MDP. Let $q: S \times A \rightarrow \mathbb{R}$ be an arbitrary function and let π be the greedy policy associated with q , then: $\|q^* - q^\pi\|_\infty \leq \frac{2\gamma}{1-\gamma} \|q^* - q\|_\infty$

q^* is the optimal value function. q^π is the greedy policy. So $T^\pi q = T^* q$.

51. Contextual bandit: For each round t , environment reveals a context $x_t \in \mathcal{X}$, learn chooses an action a_t and receives reward $r_t \sim p(\cdot | x_t, a_t)$

$$\text{We need to minimise regret: } \text{Regret}(T) = \sum_{t=1}^T (\max_a \mathbb{E}[r_t | x_t, a] - \mathbb{E}[r_t])$$

After seeing r_t , the round ends: there is no state transition. to influence future contexts. oracle. (always picks the best action on context).

We want to estimate $q(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$

52. Linear Value function approximation:

$$V_w(s) = w^T X(s), \quad L(w) = \mathbb{E}_{\pi} [(V_w(s) - V_{\pi}(s))^2]$$

Stochastic gradient descent

$$\nabla_w V_w(s_t) = X_t \Rightarrow \Delta w = \underbrace{\alpha}_{\text{step size}} \underbrace{(V_{\pi}(s_t) - V_w(s_t))}_{\text{prediction error}} \underbrace{X_t}_{\text{feature vector}}$$

53. Table lookup features: $S = \{s_1, \dots, s_n\}$ $X(s) = (I(s=s_1), \dots, I(s=s_n))^T$

54. MC: Bandits with States: $L(w) = \frac{1}{2} \mathbb{E} [(R_{t+1} - q_w(s_t, A_t))^2]$

$$w_{t+1} = w_t - \alpha \nabla_{w_t} L(w_t) = w_t + \alpha \mathbb{E} [(R_{t+1} - q_{w_t}(s_t, A_t)) \nabla_{w_t} q_{w_t}(s_t, A_t)]$$

55. Monte Carlo policy evaluation: $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t} R_T$

$$V_{\pi}(s) = \mathbb{E}[G_t | s_t = s, \pi] \quad \text{Sample average return instead of expected return.}$$

returns are well-defined until an episode ends.

56. Temporal difference learning by sampling Bellman Equations.

$$V_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma V_{\pi}(s_{t+1}) | s_t = s] \Rightarrow \text{sample } V_{t+1}(s_t) = R_{t+1} + \gamma V_t(s_{t+1})$$

$$\text{Take small step: } V_{t+1}(s_t) = V_t(s_t) + \alpha_t (\underbrace{R_{t+1} + \gamma V_t(s_{t+1})}_{\text{target}} - V_t(s_t))$$

57. Prediction setting: learn V_{π} online under policy π (we know π)

$$\text{MC: } V_{t+1}(s_t) = V_t(s_t) + \alpha (G_t - V_t(s_t))$$

$$\text{TD: } V_{t+1}(s_t) \leftarrow V_t(s_t) + \alpha (\underbrace{R_{t+1} + \gamma V_t(s_{t+1})}_{\text{TD error: } \delta_t} - V_t(s_t))$$

$$58. \quad q_{t+1}(s_t, A_t) \leftarrow q_t(s_t, A_t) + \alpha (R_{t+1} + \gamma q_t(s_{t+1}, A_{t+1}) - q_t(s_t, A_t))$$

$$\text{since } q_{t+1}(s_t, A_t) = \mathbb{E}[R_{t+1} + \gamma q_t(s_{t+1}, A_{t+1}) | s_t, A_t]$$

This is SARSA $(s_t, A_t, R_{t+1}, s_{t+1}, A_{t+1})$.

59. MC return is unbiased. TD target is biased estimate of $V_{\pi}(s_t)$ (unless $V_t(s_{t+1}) = V_{\pi}(s_{t+1})$)
MC has high variance. TD has lower variance.

60. In tabular case, both MC and TD will converge: $V_t \rightarrow V_{\pi}$.

61. Difference in Batch learning: MC converges to best mean-squared fit for the observed returns
 $\sum_{k=1}^K \sum_{t=1}^T (G_t^k - V(s_t^k))^2$ because gradient is $-(G_t - V(s)) \Rightarrow$ MC update.

TD converges to solution of max likelihood Markov model, given the data

TD() exploits Markov property. MC does not exploit Markov property.

Can help in fully-observable env. Can help in partially-observable env.

with finite data, or with function approximation, the solution may differ.

$$62. \quad n\text{-step return: } G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(s_{t+n}) = R_{t+1} + \gamma G_{t+1}^{(n-1)}$$

$$\begin{cases} \text{TD: } n=1 \\ \text{MC: } n=0 \end{cases}$$

$$63. \quad \text{Multi step TD: } V(s_t) \leftarrow V(s_t) + \alpha (G_t^{(n)} - V(s_t))$$

64. MC requires small learning rate. TD requires larger learning rate.

$$65. \quad \text{Mixing multi-step returns: } G_t^{\lambda} = R_{t+1} + \gamma((1-\lambda)V(s_{t+1}) + \lambda G_{t+1}^{\lambda})$$

$$\Rightarrow G_t^{\lambda} = \sum_{n=1}^{\infty} (1-\lambda) \lambda^{n-1} G_t^{(n)}, \quad \sum_{n=1}^{\infty} (1-\lambda) \lambda^{n-1} = 1$$

$$\begin{cases} \text{TD: } \lambda=0 \\ \text{MC: } \lambda=1 \end{cases}$$

Intuition: $\frac{1}{1-\lambda}$ is the horizon, $\lambda=0.9 \approx n=10$.

Geometric pmf (success rate = $1-\lambda$).

66. Eligibility traces: $V_w(s) = w^T X(s)$, so $\Delta w_t = \alpha (G_t - V(s_t)) X_t$

$$\text{MC update all states in episode } k \text{ at once. } \Delta w_k = \sum_{t=0}^{T-1} \alpha (G_t - V(s_t)) X_t$$

$$G_t - V(s_t) = R_{t+1} + \gamma G_{t+1} - V(s_t) = R_{t+1} + \gamma V(s_{t+1}) - V(s_t) + \gamma (G_{t+1} - V(s_{t+1}))$$

$$\Delta w_k = \sum_{t=0}^{T-1} \alpha (G_t - V(s_t)) X_t = \sum_{t=0}^{T-1} \alpha \left(\sum_{m=t}^{T-1} \gamma^{m-t} \delta_m \right) X_t = \sum_{m=0}^{T-1} \alpha \sum_{t=0}^m \gamma^{m-t} \delta_m X_t = \sum_{m=0}^{T-1} \alpha \delta_m \underbrace{\sum_{t=0}^m \gamma^{m-t} X_t}_{\equiv e_m} = \sum_{t=0}^{T-1} \alpha \delta_t e_t$$

$$\Delta W_k = \sum_{t=0}^{T-1} \alpha \delta_t e_t \quad e_t = \sigma e_{t-1} + x_t \quad e_0 = x_0. \quad e_t \text{ is called an eligibility trace.}$$

67. Mixed multi-step return $G_t^\lambda = R_{t+1} + \gamma (1-\lambda) V(S_{t+1}) + \lambda G_{t+1}^\lambda$ $G_t^\lambda = \sum_{k=0}^{T-t} \lambda^k \gamma^k S_{t+k} \Rightarrow e_t = \gamma \lambda e_{t-1} + x_t$

68. Greedy policy improvement over $V(S)$ requires model of MDP

$$\pi'(S) = \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) | S_t = S, A_t = a] \leftarrow \text{this is not model free.}$$

but over $q(S,a)$ is model free. $\pi'(S) = \operatorname{argmax}_a q(S,a)$

69. Model free control = Repeat:

- sample episode $1, \dots, k, \dots$ using $\pi = \{S_t, A_t, R_t, \dots, S_T\} \sim \pi$

- For each state S_t and action A_t in the episode,

$$q(S_t, A_t) \leftarrow q(S_t, A_t) + \alpha_t (G_t - q(S_t, A_t)).$$

- Improve policy based on new action value function. $\epsilon \leftarrow \frac{1}{k}$, $\pi \leftarrow \epsilon$ -greedy(q).

70. Greedy in the limit with Infinite Exploration (GLIE)

- $\forall s,a \lim_{t \rightarrow \infty} N_t(s,a) = \infty$, all state-action pairs are explored infinitely many times.

- policy converges to a greedy policy $\lim_{t \rightarrow \infty} \pi_t(a|s) = I(a = \operatorname{argmax}_{a'} q_t(s,a'))$.

for example: ϵ -greedy with $\epsilon_t = \frac{1}{t}$

- $\epsilon_t \downarrow 0$ (becomes greedy) - $\sum_t \epsilon_t = \infty$ (explore forever)

71. GLIE model free control converges to the optimal action-value function, $q_t \rightarrow q^*$

72. In SARSA, everytime we update Q value, we choose a policy from ϵ -greedy policy. There is no explicit policy updating. It is implicit in the Q update.

73. Tabular SARSA converges to the optimal action-value function, $q(s,a) \rightarrow q^*(s,a)$ if policy is GLIE.

74. Q-learning: $q_{t+1}(s,a) = q_t(s,a) + \alpha_t (R_{t+1} + \gamma \max_{a'} q_t(s_{t+1}, a') - q_t(s_t, a_t))$.

75. No trivial analogue version of value iteration $v_{t+1}(s) = \max_a \mathbb{E}[R_{t+1} + \gamma v_t(s_{t+1}) | S_t = s, A_t = a]$ because sampling expectation and max is difficult.

76. On-policy learning: learn about behaviour policy π from experience sampled from π .

off-policy learning: learn about target policy π from experience sampled from μ . "What if?"

SARSA is on-policy, next action taken by μ (ϵ -greedy with current Q)

Q-learning is off-policy, updates use max over actions (greedy π) but data from an ϵ -greedy μ .

77. Off-policy learning: we evaluate target policy $\pi(a|s)$ to compute $V_\pi(s)$ or $q_\pi(s,a)$ use $\mu(a|s)$ to generate actions.

In Q-learning, μ is ϵ -greedy while π is greedy.

78. Q-learning control algorithm: Q-learning control converges to the optimal action-value function, $q \rightarrow q^*$, as long as we take each action in each state infinitely often.

No need for greedy behaviour!

step size $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$

79. Q-learning overestimation: $\max_a q_t(s_{t+1}, a) = q_t(s_{t+1}, \operatorname{argmax}_a q_t(s_{t+1}, a))$

$$\mathbb{E}[\max_a (q^*(s,a) + \epsilon a)] \geq \max_a q^*(s,a)$$

because max is convex.

use same values to select and to evaluate.

but values are approximate $\Rightarrow$ more likely to select overestimated values.

(selection is not due to optimality, $\Rightarrow$ upward bias. but due to estimation error)

Because we are sample, $q(s,a)$ is not always the true $\mathbb{E}[G_{t+1} | S_t, A_t]$, it can be very largely deviated from mean if variance is huge.

80. Double Q-learning: store two action-value functions q and q'

$R_{t+1} + \gamma q'_t(s_{t+1}, \operatorname{argmax}_a q_t(s_{t+1}, a))$ - each time pick q or q' and update either one.

$R_{t+1} + \gamma q_t(s_{t+1}, \operatorname{argmax}_a q'_t(s_{t+1}, a))$ - or we can use both to act ($\frac{q+q'}{2}$).

Converges to the optimal policy under same conditions as Q-learning.

81. Importance Sampling for off-policy Monte Carlo:

$$\frac{P(\tau_t | \pi)}{P(\tau_t | \mu)} G_t = \frac{\pi(A_t | S_t) \pi(A_{t+1} | S_{t+1})}{\mu(A_t | S_t) \mu(A_{t+1} | S_{t+1})} \dots G_t \quad (\text{requires full trajectory to be greedy, otherwise } \pi = 0 \text{ for some term}).$$

82. Importance Sampling for off-policy TD:

$$V(S_t) \leftarrow V(S_t) + \alpha \left(\frac{\pi(A_t | S_t)}{\mu(A_t | S_t)} (R_{t+1} + \gamma V(S_{t+1})) - V(S_t) \right). \quad \text{Much lower variance than MC IS.}$$

policy only need to be similar over a single step.

83. Expected SARSA: (general Q-learning).

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma \sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)).$$

Q-learning is the special case with greedy target policy π .

No importance sampling is required because we don't sample A_{t+1} , instead we take expectation of all $\sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a)$. this is deterministic given S_{t+1} and current Q table.

No random variable chosen by μ .

84. Coarse coding: feature vector $\phi(s) = [\mathbb{1}\{s \in R_1\}, \mathbb{1}\{s \in R_2\}, \dots, \mathbb{1}\{s \in R_m\}]^T$
 R_n 's overlap. (local generalization, uniform approximation power)

85. Function approximation update:

- MC: $\Delta W_t = \alpha (G_t - V_w(s)) \nabla_w V_w(s)$

- TD: $\Delta W_t = \alpha (R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t)) \nabla_w V_w(S_t)$

- TD(λ): $\Delta W_t = \alpha (G_t^\lambda - V_w(S_t)) \nabla_w V_w(S_t) \quad G_t^\lambda = R_{t+1} + \gamma((1-\lambda)V_w(S_{t+1}) + \lambda G_{t+1}^\lambda)$.

86. Linear MC evaluation converges to the global optimum. Non-linear value function approximation converges but perhaps to a local optimum. Because G_t is an unbiased sample of $V_\pi(s)$.

87. TD-target $R_{t+1} + \gamma V_w(S_{t+1})$ is a biased sample of true value $V_\pi(S_t)$

88. Action-value Function approximation. with State features

$$Q_w(s) = W^T X(s) \quad W \in \mathbb{R}^{m \times n} \quad Q_w(s) \in \mathbb{R}^m, \quad Q_w(s, a) = Q_w(s)[a] = W^T a X(s) \quad W^T a \text{ is a dot row of } W$$

$$\Delta W_a = \alpha (Q_\pi(s, a) - Q_w(s, a)) \nabla_w Q_w(s, a) = \alpha (Q_\pi(s, a) - Q_w(s, a)) X(s)$$

$$\forall b \neq a: \Delta W_b = 0$$

$$\Rightarrow \Delta W = \alpha (Q_\pi(s, a) - Q_w(s, a)) \underbrace{X(s)}_{\text{vector}} \underbrace{a^T}_{\text{outer product}} \quad \bar{a} = (0, \dots, 0, 1, 0, \dots, 0)^T \text{ with } \bar{a}[a] = 1, \bar{a}[b] = 0$$

In this case (state features), we use same feature for all actions given s , vary weights.

89. Another way is $Q_w(s, a) = X(s, a)^T W$, same weights, different features across state and action.

90. Action-in: $Q_w(s, a) = W^T X(s, a)$ (good for continuous actions) Action-out: $Q_w(s) = W X(s)$ (small discrete action spaces).

91. Convergence of MC: linear functions with suitably decaying step size. MC converges to.

$$W_{MC} = \mathbb{E}_\pi [X_t X_t^T]^{-1} \mathbb{E}_\pi [G_t X_t]$$

Agent state S_t does not have to be Markov.

92. Convergence of TD: $W_{TD} = \mathbb{E}_\pi [X_t (X_t - \gamma X_{t+1})^T]^{-1} \mathbb{E}_\pi [R_{t+1} X_t]$

In tabular case, TD converges to same point as MC

TD converges faster often, but still biased in linear case.

$$93. \overline{VE}(W_{TD}) = \|V_\pi - V_w\|_{\mathcal{A}}^2 \leq \frac{1}{1-\gamma} \overline{VE}(W_{MC}) = \frac{1}{1-\gamma} \min_w \overline{VE}(w). \quad (\text{MC minimize value error}).$$

94. TD does not always converge.

95. Deadly triad: - bootstrapping - off-policy learning - function approximation.

96. TD: $\Delta w_t = \alpha \delta_t \nabla w(S_t)$ where $\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t)$ ignore dependence of $V_w(S_{t+1})$ on w
 Alternative = Bellman residual gradient. loss $\mathbb{E}[\delta_t^2]$, update: $\Delta w_t = \alpha \delta_t \nabla w(V_w(S_t) - \gamma V_w(S_{t+1}))$.
 work worse in practice and requires a second independent sample S'_{t+1} which could differ from S_{t+1} .
 - Bellman residuals smooth, whereas TD methods predict. (can't use online).

97. Convergence:

On/off-Policy	Algo	Table Lookup	Linear	Non-linear
on policy	MC TD	✓ ✓	✓ ✓	✓ ✗
off policy	MC TD	✓ ✓	✓ ✗	✓ ✗

98. Batch reinforcement learning

- least squares temporal difference (LSTD): find w to best fit linear value function $V_w(s) = w^T \phi(s)$

$$w_{LSTD} = \left(\sum_{i=0}^t \phi_i (\phi_i - \gamma \phi_{i+1})^T \right)^{-1} \left(\sum_{i=0}^t R_{i+1} \phi_i \right)$$

$$= A_t^{-1} b_t$$

update A_t^{-1} , b_t incrementally online:

- naive approach ($O(n^3)$) $A_{t+1} = A_t + \phi_t (\phi_t - \gamma \phi_{t+1})^T$ $b_{t+1} = b_t + R_{t+1} \phi_t$.
 - Faster approach ($O(n^2)$) directly update A^{-1} with Sherman-Morrison.

$$A_{t+1}^{-1} = A_t^{-1} - \frac{A_t^{-1} \phi_t (\phi_t - \gamma \phi_{t+1})^T A_t^{-1}}{1 + (\phi_t - \gamma \phi_{t+1})^T A_t^{-1} \phi_t}$$

In the limit, LSTD and TD converge to same fixed point.

- experience replay: sample transitions, apply stochastic gradient descent update.
 $(S_t, A_t, R_{t+1}, S_{t+1}) \leftarrow$ these 4 glued together in sampling to update.

99. A model M_θ is an approximate representation of an MDP $\langle S, A, P \rangle$
 the model directly approximates the state transitions and rewards $\hat{P}_\theta \approx P$
 $R_{t+1}, S_{t+1} \sim \hat{P}_\theta(r, s' | S_t, A_t)$

100. From experience $\{S_1, A_1, R_2, \dots, S_T\}$ to estimate a model
 supervised learning problem: $S_1, A_1 \rightarrow R_2, S_2$.
 $\vdots$
 $S_T, A_T \rightarrow R_T, S_T$

101. Expectation model: pick loss function, fit parameter η to minimize empirical loss
 If $f_\eta(s, a) = r, s'$, then we hope $s' \approx \mathbb{E}[S_{t+1} | S_t = s, A_t = a]$

102. With linear values: expectation model $f_\eta(\phi_t) = \mathbb{E}[\phi_{t+1}]$, value function $V_\theta(\phi_t) = \theta^T \phi_t$
 $\mathbb{E}[V_\theta(\phi_{t+1}) | S_t = s] = \mathbb{E}[\theta^T \phi_{t+1} | S_t = s] = \theta^T f_\eta(\phi_t) = V_\theta(f_\eta(\phi_t))$.

103. If model is also linear: $f_\eta(\phi_t) = P \phi_t$ for some matrix P , we can have $\mathbb{E}[V_\theta(\phi_{t+1}) | S_t = s] = V_\theta(\mathbb{E}[\phi_{t+1} | S_t = s])$

104. Stochastic model (generative model): $\hat{R}_{t+1}, \hat{S}_{t+1} = \hat{P}(S_t, A_t, \omega)$, ω is noise term.

105. Table Lookup models: $\hat{P}_t^a(s' | S, a) = \frac{1}{N(S, a)} \sum_{k=0}^{t-1} \mathbb{I}(S_k = S, A_k = a, S_{k+1} = s')$.
 $\mathbb{E}_{\hat{P}_t^a}[R_{t+1} | S_t = S, A_t = a] = \frac{1}{N(S, a)} \sum_{k=0}^{t-1} \mathbb{I}(S_k = S, A_k = a) R_{k+1}$.

106. In linear expectation models:

- assume feature ϕ is given, we parametrize separately rewards and transitions

- Expected next state $\hat{S}'(s, a) = T_a \phi(s)$ T_a is matrix.

- reward $\hat{r}(s, a) = w_a^T \phi(s)$ w_a is a vector.

Update w_a and T_a to minimize loss: $L(s, a, r, s') = (s' - T_a \phi(s))^2 + (r - w_a^T \phi(s))^2$. gradient descent.

107. Planning: Process of investing compute to improve values and policies without need to interact with the environment.

108. Use model to generate samples $S, R \sim \hat{P}_\theta(\cdot | S, a)$
apply model-free RL to samples

109. Can also use a learned model to solve MDP with value iteration or policy iteration.

110. How to deal with inevitable inaccuracies of a learned model?

- ① when model is wrong, use model-free RL.
- ② Reason about model uncertainty over θ (eg. Bayesian methods).
- ③ Combine model based and model free methods in a single algorithm.

111. Dyna-Q algorithm:

Initialise $Q(s, a)$ and Model (S, a) for all $s \in S$ and $a \in A(s)$

Do forever:

- (a) $s \leftarrow$ current state
- (b) $a \leftarrow \epsilon$ -greedy(s, Q)
- (c) get s', r from the env.
- (d) $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$.
- (e) Model $(s, a) \leftarrow s', r$ (learn model)
- (f) repeat N times:

random (s, a) previously seen:

$s', r \leftarrow$ Model (s, a)

$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$.

} learn from experience

} planning

112. Experience replay with Dyna:

sample (s, a) and infering r, s' from the model.

- plan for action selection: query a model for an action ~~that~~ could take in the future.
- counterfactual planning: query a model for an action that you ~~could~~ have taken in the past, but did not.

- backwards planning (model inverse dynamics for all predecessor pairs (s', a') that could lead to s).

- jumpy planning (planning at different timescales instead of step by step)

113. Querying a replay buffer is very cheap. Generating samples from a learned model can be expensive.

114. Prediction via Monte Carlo Simulation.

Given a parametrized model M_θ and a simulated policy π

simulate K episodes from current state S_t $\{S_t^k = S_t, A_t^k, R_{t+1}^k, S_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim \hat{P}_\theta, \pi$.

$$V(S_t) = \frac{1}{K} \sum_{k=1}^K G_t^k \rightsquigarrow v_\pi(S_t).$$

For each action $a \in A$, simulate K episodes $\{S_t^k = s, A_t^k = a, R_{t+1}^k, S_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim M_\theta, \pi$.

$$Q(s, a) = \frac{1}{K} \sum_{k=1}^K G_t^k \rightsquigarrow q_\pi(s, a) \quad \text{select } A_t = \underset{a \in A}{\operatorname{argmax}} Q(S_t, a).$$

115. Monte Carlo Tree Search.

- Select: until reaching a leaf node of the tree, pick actions according to $q(s, a)$.
- Expand: expand the search tree by one node.
- Rollout: rollout until the episode terminates with a fixed simulation policy (usually random).
- Update: action values $q(s, a)$ for all state-action pairs in the tree

$$Q(s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{u=t}^T \mathbb{1}\{S_u^k, A_u^k = s, a\} G_u^k \rightsquigarrow q_\pi(s, a).$$

It is an off-policy algorithm. Simulation policy μ is different from tree policy π . MCTS is a planning algorithm, it is noise value estimator. But the search grows deeper, the rollout portion becomes shorter. In the limit, the rollout segment disappears and bias vanishes anyway. We don't add importance sampling ratio.

116. Value-based: learn values, implicit policy (ϵ -greedy for example).

policy-based: No values, learn policy

Actor-critic: learn values, learn policy.

117. In MDPs, there is always an optimal deterministic policy. But most problems are not fully observable the optimal policy may then be stochastic. Search space is smoother for stochastic policies. Also provides some exploration during learning.

118. Measure quality of a policy π_0 :

- In episodic environments we can use average total return per episode
- In continuing environments we can use the average reward per step.

119. Episodic-return objective $J_G(\theta) = \mathbb{E}_{S_0 \sim d_0, \pi_0} \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] = \mathbb{E}_{S_0 \sim d_0, \pi_0} [G_0] = \mathbb{E}_{S_0 \sim d_0} [\mathbb{E}_{\pi_0} [G_0 | S_0]] = \mathbb{E}_{S_0 \sim d_0} [V_{\pi_0}(S_0)]$

120. Average-reward objective $J_R(\theta) = \mathbb{E}_{\pi_0} [R_{t+1}] = \mathbb{E}_{S_t \sim d_{\pi_0}} [\mathbb{E}_{A_t \sim \pi_0(S_t)} [R_{t+1} | S_t]] = \sum_s d_{\pi_0}(s) \sum_a \pi_0(s, a) \sum_r p(r | s, a) r$.

$d_{\pi_0}(s) = p(S_t = s | \pi)$ is the probability of being in state s in the long run. Ratio of time spent in s under π .

121. Policy gradient theorem (episodic): $J(\theta) = \mathbb{E}[G_0 | S_0 \sim d_0]$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_0} \left[\sum_{t=0}^{\infty} \gamma^t q_{\pi_0}(S_t, A_t) \nabla_{\theta} \log \pi_0(A_t | S_t) | S_0 \sim d_0 \right]$$

$$q_{\pi_0}(s, a) = \mathbb{E}_{\pi} [G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} [R_{t+1} + \gamma q_{\pi_0}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

Do not need to know MDP dynamics.

Typically we use $\Delta \theta_t = \gamma^t G_t \nabla_{\theta} \log \pi(A_t | S_t)$ and $\mathbb{E}_{\pi} [\sum_t \Delta \theta_t] = \nabla_{\theta} J_{\theta}(\pi)$.

we also ignore γ^t , just use $\Delta \theta_t = G_t \nabla_{\theta} \log \pi(A_t | S_t)$.

122. Policy gradient theorem (average reward): $J(\theta) = \mathbb{E}[R | \pi]$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [q_{\pi_0}(S_t, A_t) \nabla_{\theta} \log \pi_0(A_t | S_t)] \quad \text{expectation is over states and actions}$$

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [R_{t+1} - \rho + q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \quad \rho = \mathbb{E}_{\pi} [R_{t+1}]$$

equivalently: $\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [R_{t+1} \underbrace{\sum_{n=0}^{\infty} \nabla_{\theta} \log \pi_0(A_{t+n} | S_{t+n})}_{\text{sum of past}}]$ expectation is over states and actions.

123. Actor-critics

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t (q_{\pi_0}(S_t, A_t) - V_{\pi}(S_t)) \nabla_{\theta} \log \pi_0(S_t | A_t) \right] \quad \text{reduce variance}$$

estimate $V_w(s) \approx V_{\pi}(s)$, sample $q_{\pi_0}(S_t, A_t) \approx G_t$ or bootstrapping $R_{t+1} + \gamma V_w(S_{t+1})$. further reduce variance.

Critic: update parameters w of V_w by TD or MC.

Actor: update θ by policy gradient.

Initialize s, θ, w

for $t=0, 1, 2, \dots$ do

Sample $A_t \sim \pi_0(S_t)$

Sample R_{t+1}, S_{t+1} .

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t)$$

(one step TD error).

$$w \leftarrow w + \beta \delta_t \nabla_w V_w(S_t)$$

(TD update).

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_{\theta} \log \pi_0(A_t | S_t)$$

(policy gradient, ignore γ^t term).

} $\rightarrow$ bad policy lead to bad data.

124. Increase robustness with trust regions.

limit the different between subsequent policies.

$$KL(\pi_{old} || \pi_{\theta}) = \mathbb{E}_s \left[\int \pi_{old}(a|s) \log \frac{\pi_{old}(a|s)}{\pi_{\theta}(a|s)} da \right].$$

$$\text{maximise } J(\theta) - \eta KL(\pi_{old} || \pi_{\theta}).$$

125. Continuous space policy: Gaussian policy $A_t \sim N(\mu_{\theta}(S_t), \sigma^2)$ $\mu_{\theta}(S_t)$ is parametrised.

$$\nabla_{\theta} \log \pi_{\theta}(S, a) = \frac{A_t - \mu_{\theta}(S_t)}{\sigma^2} \nabla \mu_{\theta}(S).$$

If return is positive $A_t - \mu_{\theta}(S_t) > 0$, μ_{θ} move towards action
If return is negative, μ_{θ} move away.

126. Gradient ascent on value.

- estimate $q_w \approx q_{\pi}$ with eg. SARSA (value parametrized by π)
- define deterministic actor $A_t = \pi_{\theta}(S_t)$
- policy improvement by gradient ascent on the value (critic).

} Deterministic policy gradient
it is a form of policy iteration.

$$\Delta \theta \propto \frac{\partial Q_{\pi}(S, a)}{\partial \theta} = \frac{\partial Q_{\pi}(S, \pi_{\theta}(S))}{\partial \pi_{\theta}(S)} \frac{\partial \pi_{\theta}(S)}{\partial \theta}.$$

127. Continuous actor-critic learning automation (CACA)

- ① $a_t = \text{Actor}_{\theta}(S_t)$ (deterministic mean action produced by actor network).
- ② $A_t \sim \pi(\cdot | S_t, a_t)$ eg. $A_t \sim N(a_t, \Sigma)$ (explore, $\nabla_{\theta} \log \pi_{\theta}(S, a) \propto (A_t - a_t) \nabla_{\theta} a_t$).
- ③ TD error $\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t)$.
- ④ update $V_w(S_t)$
- ⑤ $\delta_t > 0$, update $\text{Actor}_{\theta}(S_t)$ towards A_t $\theta_{t+1} \leftarrow \theta_t + \beta (A_t - a_t) \nabla_{\theta} a_t$
 $\delta_t < 0$, do not update.

Do not update away from bad actions.

128. Consider an MDP let q_k be the value function returned by approximate value iteration after k steps and let π_k be its corresponding greedy policy, then.

$$\|q^* - q_{\pi_k}\|_{\infty} \leq \frac{2\gamma}{(1-\gamma)^2} \max_{0 \leq k \leq n} \|T^* q_k - AT^* q_k\|_{\infty} + \frac{2\gamma^{n+1}}{1-\gamma} \sum_{\text{initial error}}.$$

$\sum_0 = \|q^* - q_0\|_{\infty}$. T^* is Bellman optimality operator of this MDP

$q_{k+1} = AT^* q_k$ q_{π_k} is the greedy policy q_{π_k}

129. $n \rightarrow \infty$ $\|q^* - q_{\pi_k}\|_{\infty} \leq \frac{2\gamma}{(1-\gamma)^2} \max_{0 \leq k \leq n} \|T^* q_k - AT^* q_k\|_{\infty}$ even we start from q^* , $\|Aq^* - q^*\|_{\infty} > 0$
in general, we can still move away. The error is highly dependent on the approximation error.

130. If $A = \Pi_{\mathcal{F}}$ is a projection operation in L_{∞} onto hypothesis space $\mathcal{F}$.

$$\Pi_{\mathcal{F}} g := \arg \min_{f \in \mathcal{F}} \|g - f\|_{\infty}. \quad q_{k+1} = AT^* q_k = \Pi_{\mathcal{F}} T^* q_k = \arg \min_{f \in \mathcal{F}} \|T^* q_k - f\|_{\infty}.$$

$AT^* = \Pi_{\mathcal{F}} T^*$ is a contraction operator in $L_{\infty} \Rightarrow$ converges to its fixed point $f = \Pi_{\mathcal{F}} T^* f$

If $q^* \in \mathcal{F}$, $f = q^*$ ($\Pi_{\mathcal{F}} T^*(q^*) = \Pi_{\mathcal{F}}(T^*(q^*)) = \Pi_{\mathcal{F}} q^* = q^*$ given $q^* \in \mathcal{F}$).

131. Propose algorithm $q_{k+1} = \Pi_{\mathcal{F}} T^* q_k = \arg \min_{f \in \mathcal{F}} \|T^* q_k - f\|_{\infty}$

potential problems: P_1 : L_{∞} minimisation hard to carry out efficiently. $\rightarrow L_2$

P_2 : T^* is typically unknown and need approximation $\rightarrow$ sample to approx. T^*

132. Fitted Q-iteration (General recipe)

- Every iteration $k+1$:

$$Q_{k+1} = \arg \min_{Q \in \mathcal{F}} \frac{1}{n_{\text{samples}}} \sum_{i=1}^{n_{\text{samples}}} (\tilde{T}^* Q_k(S_t, A_t) - Q_0(S_t, A_t))^2$$

for samples $(S_t, A_t, R_{t+1}, S_{t+1}) \sim \mu, P$.

$\mathcal{F} = \mathcal{F}_0$ can be: Linear functions, neural networks, kernel functions...

Samples: Online, fixed dataset, replay memory, generative model.

Target: $\tilde{T}^* Q_k = R_{t+1} + \gamma \max_a Q_k(S_{t+1}, a)$, $\tilde{T}^* Q_{\text{target}} = \tilde{T}^* Q_0$, off-policy updates, multistep operators.

133. DQN: neural networks, replay memory, $\tilde{T}^* Q_{\text{target}} = \tilde{T}^* Q_0$

Batch RL-1: all type $\mathcal{F}_0$, Fixed dataset, $\tilde{T}^* Q_k = R_{t+1} + \gamma \max_a Q_k(S_{t+1}, a)$.

Batch RL-2: all type $\mathcal{F}_0$, Fixed dataset, off policy updates.

Dyna: all type $\mathcal{F}_0$, Online/replay memory, $\tilde{T}^* Q_k = R_{t+1} + \gamma \max_a Q_k(S_{t+1}, a)$.

134. Approximate policy iteration. Consider an MDP, let Q_k, π_k be value function and respectively evaluated greedy policy achieved by Approximate policy iteration at iteration k , then.

$$\limsup_{k \rightarrow \infty} \|Q^* - Q_{\pi_k}\|_{\infty} \leq \frac{2\gamma}{(1-\gamma)^2} \limsup_{k \rightarrow \infty} \|Q_{\pi_k} - Q_k\|_{\infty}$$

approximation error at iter k .

In PI, we have $Q_i = Q_{\pi_i}$, but in API, $Q_i = A Q_{\pi_i}$.

135. TD(λ) with Linear approximation ($\mathcal{F}_\phi = \{Q_w(S, a) = w^T \phi(S, a) \mid w \in B\}$).

$$\|Q_w^* - Q_\pi\|_{2, \mu^\pi} \leq \frac{1-\lambda\gamma}{1-\gamma} \inf_w \|Q_w - Q_\pi\|_{2, \mu^\pi}$$

- $\lambda=1$ (MC case) has the tightest bound.

- if $Q_\pi \in \mathcal{F}_\phi$, $Q_w^* = Q_\pi$

- if $Q_\pi \notin \mathcal{F}_\phi$, $Q_w^* \neq \inf_w \|Q_w - Q_\pi\|_{2, \mu^\pi}$

136. Off policy learning is important to learn about hypothetical, counterfactual events

137. MC trajectory $\tau_t = \{S_t, A_t, R_{t+1}, \dots, S_T\}$

$$\hat{G}_t \equiv \frac{\pi(A_t|S_t) \dots \pi(A_{T-1}|S_{T-1})}{\mu(A_t|S_t) \dots \mu(A_{T-1}|S_{T-1})} G_t \Rightarrow \mathbb{E}[\hat{G}_t | \mu] = \mathbb{E}[G_t | \pi]$$

138. Issue with off-policy learning:

- high variance (especially when using multi-step updates)

- divergent and inefficient learning (especially when using one-step updates).

139. high variance: $\mathbb{E}[\frac{\pi(A_t|S)}{\mu(A_t|S)} R_{t+1} \mid A_t \sim \mu] = \mathbb{E}[R_{t+1} \mid A_t \sim \pi]$.

140. Variance reduction: Per-decision importance sampling. Control variates, adaptive bootstrapping.

141. Per-decision importance sampling:

$$P_t \equiv \frac{\pi(A_t|S_t)}{\mu(A_t|S_t)} \quad P_{t:t+1} \equiv \prod_{k=t}^{t+1} P_k = \prod_{k=t}^{t+1} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}$$

$$\mathbb{E}[P_{t:T-1} G_t | \mu] = \mathbb{E}\left[\sum_{k=t}^{T-1} P_{t:k-1} \gamma^{k-t} R_{k+1} \mid \mu\right] = \mathbb{E}\left[\sum_{k=t}^{T-1} P_{t:k} \gamma^{k-t} R_{k+1} \mid \mu\right] \text{ (earlier rewards do not depend on later } P)$$

$$G_t^P = P_t (R_{t+1} + \gamma G_{t+1}^P) = P_t R_{t+1} + P_{t:t+1} \gamma R_{t+2} + P_{t:t+2} \gamma^2 R_{t+3} + \dots$$

compare to $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \dots = R_{t+1} + \gamma G_{t+1}$

$G_t^P = P_t (R_{t+1} + \gamma G_{t+1}^P) \Rightarrow$ unbiased estimator for V_π

To learn action values Q_π , $G_t^P = R_{t+1} + \gamma P_{t+1} G_{t+1}^P$ We have conditioned on A_t so no need to correct it.

For terminal state: state value estimator V_π : $G_{T-1}^P = P_{T-1} (R_T + \gamma G_T^P) = P_{T-1} R_T$. (we still need to choose A_{T-1})
action value estimator Q_π : $G_{T-1}^P = R_T + \gamma P_T G_T^P = R_T$ (we have fixed A_{T-1} , so no need to correct).

142. Control Variates.

$$P_t G_t^\lambda - V(S_t) = P_t (R_{t+1} + \gamma((1-\lambda)V(S_{t+1}) + \lambda G_{t+1}^\lambda)) - V(S_t) \\ = P_t (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)) - (1-P_t)V(S_t) + P_t \gamma \lambda (G_{t+1}^\lambda - V(S_{t+1})).$$

add $(1-P_t)V(S_t)$ and doesn't change expectation:

$$P_t (G_t^\lambda - V(S_t)) = P_t (R_{t+1} + \gamma V(S_{t+1}) - V(S_t)) + P_t \gamma \lambda (G_{t+1}^\lambda - V(S_{t+1})).$$

$$P_t \delta_t^\lambda = P_t \delta_t + \gamma \lambda P_t \delta_{t+1}^\lambda \Rightarrow \delta_t^\lambda = P_t (\delta_t + \gamma \lambda \delta_{t+1}^\lambda). \quad \begin{matrix} \delta_t = TD(0) \text{ error} \\ \delta_t^\lambda = TD(\lambda) \text{ error} \end{matrix}$$

$$\mathbb{E}[\delta_t^\lambda | \mathcal{M}] = \mathbb{E}[P_t G_t^\lambda - P_t V(S_t) | \mathcal{M}] = \mathbb{E}[P_t G_t^\lambda | \mathcal{M}] - V(S_t). \\ = \mathbb{E}[G_t^\lambda - V(S_t) | \mathcal{M}]. \quad G_t^\lambda = P_t (R_{t+1} + \gamma((1-\lambda)V(S_{t+1}) + \lambda G_{t+1}^\lambda))$$

$$\text{Terminal state: } G_{T-1}^\lambda = P_{T-1} R_T, \quad \delta_{T-1}^\lambda = P_{T-1} (G_{T-1}^\lambda - V(S_{T-1})) = P_{T-1} (R_T - V(S_{T-1})).$$

$$\text{To learn action values: } G_t^\lambda = R_{t+1} + \gamma((1-\lambda) \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) + \lambda P_{t+1} G_{t+1}^\lambda).$$

P_t only appears around G_{t+1}^λ as R_{t+1} has conditioned on A_t , $\sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a)$ is expectation of action value and is independent of any single A_{t+1} .

$$\lambda = 0 \text{ (TD case): } G_t^\lambda = R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a). \quad (\text{no more importance sampling} \rightarrow \text{low variance but } \text{deadly triad})$$

$$\lambda = 1 \text{ (MC case): } G_t^\lambda = R_{t+1} + \gamma P_{t+1} G_{t+1}^\lambda \quad (\text{same as per-decision importance sampling for action value})$$

bootstrap too much $\Rightarrow$ deadly triad bootstrap too little $\Rightarrow$ high variance.

143. Adaptive bootstrapping:

$$G_t^{(n)} := R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

$$G_t^{(n)} - V(S_t) = \underbrace{R_{t+1} - V(S_t)}_{\delta_t} + \gamma \underbrace{R_{t+2} - V(S_{t+1})}_{\delta_{t+1}} + \gamma^2 \underbrace{R_{t+3} - V(S_{t+2})}_{\delta_{t+2}} + \dots + \gamma^{n-1} \underbrace{R_{t+n} - V(S_{t+n-1})}_{\delta_{t+n-1}} + \gamma^n \underbrace{V(S_{t+n}) - V(S_t)}_{\delta_{t+n}} = \sum_{k=0}^{n-1} \gamma^k \delta_{t+k}$$

$$\tilde{G}_t^{(n)} = P_{t+n-1} G_t^{(n)} \quad \tilde{G}_t^{(n)} - V(S_t) = \sum_{k=0}^{n-1} \left(\prod_{i=t}^{t+k-1} P_i \right) \gamma^k \delta_{t+k}$$

$$G_t^\lambda - V(S_t) = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} (\tilde{G}_t^{(n)} - V(S_t)) = \sum_{k=0}^{\infty} \left[\prod_{i=t}^{t+k-1} P_i \lambda_i \right] \gamma^k \delta_{t+k}.$$

$$\text{let } \delta_t^\lambda = G_t^\lambda - V(S_t), \text{ we have } \delta_t^\lambda = \lambda_t P_t (\delta_t + \gamma \delta_{t+1}^\lambda) \quad \delta_{T-1}^\lambda = P_{T-1} R_T - V(S_{T-1}).$$

note this definition of δ_t^λ is not same as control variate. In control variate we have $\delta_t^\lambda := P_t (G_t^\lambda - V(S_t))$

Pick λ_t separately for each step so that $\lambda_t P_t \leq 1 \Rightarrow \lambda_t = \min(1, \frac{1}{P_t})$.

no large $\frac{\pi_t}{\lambda_t}$ ratio, no too off-policy.

When we are too off-policy, truncate the sum of errors and just do bootstrapping there.

144. Tree backup:

$$Q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q_\pi(S_{t+1}, a) | A_t = a, S_t = s]$$

$$\text{we sample } G_t = R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a|S_{t+1}) Q_\pi(S_{t+1}, a) + \gamma \pi(A_{t+1}|S_{t+1}) G_{t+1}. \quad \text{unbiased and low variance}$$

$$\text{Similar to } G_t = R_{t+1} + \gamma((1-\lambda)V(S_{t+1}) + \lambda G_{t+1}). \quad \pi(A_{t+1}|S_{t+1}) \text{ play a role similar to } \lambda.$$

If π is greedy, $G_t = R_{t+1} + \gamma G_{t+1}$ (MC).

If π is not greedy, $\prod_{i=t}^k \pi(A_{i+1}, S_{i+1})$ will shrink exponentially fast, the sampling term has very small weight so essentially it is bootstrapping Q_π .

145. DQN (double-Q network): parameter θ and target parameter $\theta^- \leftarrow \theta$, replay buffer.

for episode 1 to M do

initialize state s

for $t=1$ to T do

$a \sim \pi$ -greedy, execute a and receive r and s'

store (s, a, r, s') into buffer $\mathcal{D}$

$s \leftarrow s'$

sample min-batch (s_i, a_i, r_i, s'_i) from $\mathcal{D}$

targets $y_i = r_i + \gamma \max_{a'} Q_{\theta^-}(s'_i, a')$

compute loss: $L(\theta) = \frac{1}{|\text{batch}|} \sum (y_i - Q_\theta(s_i, a_i))^2$

perform step on θ

every C step, update $\theta^- \leftarrow \theta$.

146. Prioritized replay:

DQN uses uniform replay samples.

We can prioritise transitions on which we learn much.

priority of sample $i = |\delta_i|$ δ_i is TD error.

147. Multi-step Q-learning:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \max_{a'} Q_{\theta}(S_{t+n}, a'). \text{ partially on policy, partially off}$$

$$\Delta \theta = \alpha (G_t^{(n)} - Q_{\theta}(S_t, A_t)) \nabla_{\theta} Q_{\theta}(S_t, A_t).$$

148. General value functions

$$Q_{C, \gamma, \pi}(s, a) = \mathbb{E}[C_{t+1} + \gamma C_{t+2} + \gamma^2 C_{t+3} + \dots | S_t = s, A_{t+1} \sim \pi(S_{t+1})]$$

$$C_t = C(S_t), \quad \gamma_t = \gamma(S_t)$$

$C: S \rightarrow \mathbb{R}$ is the cumulant $\gamma: S \rightarrow \mathbb{R}$ is the discount $\pi: S \rightarrow \mathcal{A}$ is the target policy.

- policy evaluation: $C_t = R_t$

- reward prediction: $C_t = R_t, \gamma = 0 \quad \mathbb{E}[R_{t+1} | S]$

- next state prediction: $\{C_t^i = S_t^i\}_i, \gamma = 0$ or $\{C_t^i = \phi(S_t^i)\}_i, \gamma = 0. \quad \mathbb{E}[S_{t+1} | S]$

149. Adaptive target normalisation:

$$\text{Target: } T_{t+1} = R_{t+1} + \gamma \max_a Q_{\theta}(S_{t+1}, a)$$

$$\text{update normalization parameters: } \mu_{t+1} = \mu_t + \eta (T_{t+1} - \mu_t) \quad \sigma_{t+1}^2 = \sigma_t^2 + \eta (T_{t+1}^2 - \sigma_t^2)$$

$$\Delta \theta_t \propto \left(\frac{T_{t+1} - \mu_{t+1}}{\sigma_{t+1}} - \tilde{Q}_{\theta}(S_t, A_t) \right) \nabla_{\theta} \tilde{Q}_{\theta}(S_t, A_t) \text{ where } \tilde{Q}_{\theta}(s, a) = \sigma_t \tilde{Q}_{\theta}(s, a) + \mu_t.$$

but everytime $Q_{\theta}(s, a) = \sigma_t \tilde{Q}_{\theta}(s, a) + \mu_t$ changes all the states, even ones we haven't touched.

trick: $\tilde{Q}_{\theta}(s) = W\phi(s) + b$, $Q_{\theta}(s) = \sigma_t(W\phi(s) + b) + \mu_t$, after update σ_{t+1} and μ_{t+1} ,

we have $Q_{\text{new}}(s) = \sigma_{t+1}(W\phi(s) + b) + \mu_{t+1}$ which change the untouched value.

so we solve new W' and b' such that $\sigma_{t+1}(W'\phi(s) + b') + \mu_{t+1} = \sigma_t(W\phi(s) + b) + \mu_t$ for $\forall s$.

$$W' = \frac{\sigma_t}{\sigma_{t+1}} W \quad b' = \frac{\sigma_t b_t + \mu_t - \mu_{t+1}}{\sigma_{t+1}}, \text{ then proceed with gradient update.}$$

150. Distributional RL: learn the distribution of returns instead of just approximating expected value.

- categorical DQN: $Z = (-10, -9.9, \dots, 9.9, 10)^T$, each point assign probability $p_{\theta}^i(S_t, A_t)$.

approximate distribution of the return s and a is the tuple $(Z, p_{\theta}(S, a))$.

$$Z^T p_{\theta}(S, a) \approx Q(S, a)$$

- categorical return distributions:

$$\textcircled{1} \text{ find max action } a^* = \arg \max_a \underbrace{Z^T p_{\theta}(S_{t+1}, a)}_{\mathbb{E}[Z]}$$

$$Z \text{ is random variable for discounted return } p_{\theta}(s, a) = [p_1, \dots, p_n]^T \quad \sum_j p_j = 1$$

② update support $Z' = R_{t+1} + \gamma Z$ Z' has different support from Z

③ project $(Z', p_{\theta}(S_{t+1}, a^*))$ onto support of Z , $d^i(Z, p^i) = \Pi(Z', p_{\theta}(S_{t+1}, a^*))$

④ fit $p_{\theta}(S, a)$ towards p^i by minimising $KL(d^i || d) = \sum_i p_i' \frac{\log p_i'}{\log p_{\theta}(S_t, A_t)}$